

How to Cancel a Long-Running Task Using Cancellation Token in C#?

In this article, I will discuss **How to Cancel a Long-Running Task in C# using Cancellation Token in C#** with Examples. Please read our previous article discussing [How to Limit the Number of Concurrent Tasks in C# using SemaphoreSlim](#) with Examples. At the end of this article, you will understand the following pointers in detail:

1. [How to Cancel a Long-Running Task in C#?](#)
2. [How to use a Cancellation Token to Cancel a Task in C#?](#)
3. [Constructors, Properties, and Methods of CancellationTokenSource Class in C#](#)
4. [How to Create and Use a Cancellation Token in C#?](#)
5. [Example to Understand Cancellation Token in C#](#)
6. [When to Use Cancellation Token to Cancel a Task in C#?](#)
7. [Realtime Example to Understand Cancellation Token in C#](#)

How to Cancel a Long-Running Task in C#?

When we run a long task, providing users with some mechanism to cancel the task is a good practice. The .NET Framework provides the Cancellation Token, using which we can cancel a task. Cancelling a Task in C# using a CancellationToken is a powerful way to terminate asynchronous operations.

You can use a CancellationToken to signal to a running task that it should stop executing. You can cancel a long-running Task in C# by using a CancellationToken and periodically checking whether cancellation has been requested within the Task's logic.

How to use a Cancellation Token to Cancel a Task in C#?

Let us see the steps or procedure to cancel a long-running task using Cancellation Token. So, what we will do here is, we are going to generate a cancellation token, and we will pass that token to the task that we want to cancel. Before doing the practical implementation, let us first understand the CancellationTokenSource class.

If you go to the definition of **CancellationTokenSource** class, you will find the following. It is a class implementing the IDisposable interface. This CancellationTokenSource Signals to a CancellationToken that it should be canceled.

```

namespace System.Threading
{
    ...public class CancellationTokenSource : IDisposable
    {
        ...public CancellationTokenSource();
        ...public CancellationTokenSource(TimeSpan delay);
        ...public CancellationTokenSource(int millisecondsDelay);

        ...public bool IsCancellationRequested { get; }
        ...public CancellationToken Token { get; }

        ...public static CancellationTokenSource CreateLinkedTokenSource(CancellationToken token1, CancellationToken token2);
        ...public static CancellationTokenSource CreateLinkedTokenSource(params CancellationToken[] tokens);
        ...public void Cancel();
        ...public void Cancel(bool throwOnFirstException);
        ...public void CancelAfter(TimeSpan delay);
        ...public void CancelAfter(int millisecondsDelay);
        ...public void Dispose();
        ...protected virtual void Dispose(bool disposing);
    }
}

```

Constructors of CancellationTokenSource Class in C#:

The CancellationTokenSource class provides the following three constructors to create an instance of the CancellationTokenSource class.

1. **CancellationTokenSource()**: It Initializes a new instance of the CancellationTokenSource class.
2. **CancellationTokenSource(TimeSpan delay)**: It initializes a new instance of the CancellationTokenSource class that will be canceled after the specified time span. Here, the parameter delay specifies the time interval to wait before canceling this CancellationTokenSource. It will throw ArgumentOutOfRangeException if delay.System.TimeSpan.TotalMilliseconds is less than -1 or greater than System.Int32.MaxValue.
3. **CancellationTokenSource(int millisecondsDelay)**: It initializes a new instance of the CancellationTokenSource class that will be canceled after the specified delay in milliseconds. Here, the parameter millisecondsDelay specifies the time interval in milliseconds to wait before canceling this System.Threading.CancellationTokenSource. It will throw ArgumentOutOfRangeException if millisecondsDelay is less than -1.

Properties of CancellationTokenSource class in C#:

The CancellationTokenSource class in C# provides the following two properties:

1. **public bool IsCancellationRequested { get; }**: It gets whether cancellation has been requested for this CancellationTokenSource. It returns true if the cancellation has been requested for this CancellationTokenSource; otherwise, false.
2. **public CancellationToken Token { get; }**: It gets the CancellationToken associated with the CancellationTokenSource. It returns the CancellationToken associated with this CancellationTokenSource. It will throw ObjectDisposedException if the token source has been disposed.

Methods of CancellationTokenSource class in C#:

The CancellationTokenSource class provides the following methods:

1. **Cancel():** It communicates a request for cancellation.
2. **Cancel(bool throwOnFirstException):** It communicates a request for cancellation and specifies whether remaining callbacks and cancellable operations should be processed if an exception occurs. Here, the parameter `throwOnFirstException` specifies true if exceptions should immediately propagate; otherwise, false.
3. **CancelAfter(TimeSpan delay):** It schedules a cancel operation on the `CancellationTokenSource` after the specified time span. Here, the parameter `delay` specifies the time span to wait before canceling this `CancellationTokenSource`.
4. **CancelAfter(int millisecondsDelay):** It schedules a cancel operation on this `CancellationTokenSource` after the specified number of milliseconds. Here, the parameter `millisecondsDelay` specifies the time span to wait before canceling this `System.Threading.CancellationTokenSource`.
5. **Dispose():** It releases all resources used by the current instance of the `CancellationTokenSource` class.

How to Create and Use a Cancellation Token in C#?

First, we need to create an instance of the `CancellationTokenSource` class as follows. This `CancellationTokenSource` will create a `CancellationToken` that can be passed to the task.

```
CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
```

Then, we need to set the time interval, i.e., when this token will cancel the task execution. Here, we need to call the `CancelAfter` method of the `CancellationTokenSource` instance, and we need to specify the time in milliseconds as follows. It will cancel the task after 5 seconds as we specify 5000 milliseconds.

```
cancellationTokenSource.CancelAfter(5000);
```

Next, our async method should accept the `CancellationToken` as a parameter. If you go to the definition of the `CancellationToken` class, you will see that this class has one property called `IsCancellationRequested`, which returns true if the cancellation has been requested for this token; otherwise, it is false. If it returns true, we must stop the execution and return. But as a standard, we need to throw `TaskCanceledException`. For a better understanding, please have a look at the below image.

CancellationToken as a parameter

```
public static async Task LongRunningTask( CancellationToken token )
{
    for (int i = 1; i < count; i++)
    {
        //Doing Sometask

        if (token.IsCancellationRequested)
        {
            throw new TaskCanceledException();
        }
    }
}
```

Check if Cancellation Requested
If Yes, throw TaskCanceledException

Next, while calling the LongRunningTask method, we must pass the Cancellation Token. If you remember, the CancellationTokenSource class has one property called **Token**, and that property return type is CancellationToken, i.e., if we call the Token property on the CancellationTokenSource instance, then we will get the CancellationToken and that cancellation token, we need to pass to the LongRunningTask method as shown in the below image. Further, if you remember, the LongRunningTask method throws **TaskCanceledException** when the task is canceled, and hence, we need to use the try-catch block to handle the exception, as shown in the image below.

Creating an instance of CancellationTokenSource
It will generate the Cancellation Token

```
CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
```

```
cancellationTokenSource.CancelAfter(5000);
```

Setting the Time Interval as 5 seconds
So, within 5 seconds, if the task not completed,
then the task will be cancelled.

```
try
```

```
{
```

The Token Property of CancellationTokenSource class
returns CancellationToken

```
    await LongRunningTask(cancellationTokenSource.Token);
```

```
}
```

```
catch (TaskCanceledException ex)
```

If the Task cancelled, it will throw TaskCanceledException
which is going to be handled by this catch block

```
{
    Console.WriteLine($"{ex.Message}");
```

```
}
```

I hope you understand how to create and use Cancellation Token. Let us see an example for a better understanding.

Example to Understand Cancellation Token in C#:

```

using System;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            SomeMethod();
            Console.ReadKey();
        }
        private static async void SomeMethod()
        {
            int count = 10;
            Console.WriteLine("SomeMethod Method Started");
            CancellationTokensource cancellationTokensource = new
CancellationTokensource();
            cancellationTokensource.CancelAfter(5000);
            try
            {
                await LongRunningTask(count, cancellationTokensource.Token);
            }
            catch (TaskCanceledException ex)
            {
                Console.WriteLine($"{ex.Message}");
            }
            Console.WriteLine("\nSomeMethod Method Completed");
        }
        public static async Task LongRunningTask(int count, CancellationToken
token)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            Console.WriteLine("\nLongRunningTask Started");
            for (int i = 1; i <= count; i++)
            {
                await Task.Delay(1000);
                Console.WriteLine("LongRunningTask Processing....");
                if (token.IsCancellationRequested)
                {
                    throw new TaskCanceledException();
                }
            }
            stopwatch.Stop();
            Console.WriteLine($"LongRunningTask Took
{stopwatch.ElapsedMilliseconds / 1000.0} Seconds for Processing");
        }
    }
}

```

In the above example, we set the count variable value to 10. That means the loop inside the LongRunningTask method will execute 10 times. And inside the loop, we have delayed the execution for 1 second. That means the loop will take at least 10 seconds to execute. And we have set the cancellation token time to 5 seconds. Inside this method, we are checking whether we get the token cancellation request. If the IsCancellationRequested property returns true, 5 seconds is over, and then we throw TaskCanceledException. So, when you run the above code, you will get the following output.

```
SomeMethod Method Started

LongRunningTask Started
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
A task was canceled.

SomeMethod Method Completed
```

Now, if you set the count variable value to less than 5 and execute the code, you will see that the task is completed without throwing the TaskCanceledException.

Note: Instead of using the CancelAfter() method to set the time, you can also use the constructor, which takes time in milliseconds, or TimeSpan as an input parameter. For a better understanding, please have a look at the below image.

```
CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
cancellationTokenSource.CancelAfter(5000);
```



Both are same

```
CancellationTokenSource cancellationTokenSource = new CancellationTokenSource(5000);
```

Example:

```
using System;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

        SomeMethod();
        Console.ReadKey();
    }
    private static async void SomeMethod()
    {
        int count = 10;
        Console.WriteLine("SomeMethod Method Started");
        CancellationTokensource cancellationTokensource = new
CancellationTokensource(5000);
        //cancellationTokensource.CancelAfter(5000);
        try
        {
            await LongRunningTask(count, cancellationTokensource.Token);
        }
        catch (TaskCanceledException ex)
        {
            Console.WriteLine($"{ex.Message}");
        }
        Console.WriteLine("\nSomeMethod Method Completed");
    }
    public static async Task LongRunningTask(int count, CancellationTokensource
token)
    {
        var stopwatch = new Stopwatch();
        stopwatch.Start();
        Console.WriteLine("\nLongRunningTask Started");
        for (int i = 1; i <= count; i++)
        {
            await Task.Delay(1000);
            Console.WriteLine("LongRunningTask Processing....");
            if (token.IsCancellationRequested)
            {
                throw new TaskCanceledException();
            }
        }
        stopwatch.Stop();
        Console.WriteLine($"LongRunningTask Took
{stopwatch.ElapsedMilliseconds / 1000.0} Seconds for Processing");
    }
}
}

```

Output:

```
SomeMethod Method Started

LongRunningTask Started
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
LongRunningTask Processing....
A task was canceled.

SomeMethod Method Completed
```

When to Use Cancellation Token to Cancel a Task in C#?

In C#, you can use a `CancellationToken` to cancel a `Task` under certain conditions. `CancellationToken` is a mechanism the Task Parallel Library (TPL) provides to signal cancellation to a running `Task`. You need to use it when you want to cancel the execution of a `Task` based on some external conditions or user requests. Here are some common scenarios and approaches for using `CancellationToken` to cancel a `Task`:

User-Initiated Cancellation:

You can use `CancellationToken` to allow users to cancel long-running operations. For example, suppose you have a `Task` that performs a time-consuming computation or downloads a large file. In that case, you can periodically check a `CancellationToken` and gracefully exit the `Task` if cancellation is requested. Here's an example:

```
CancellationTokenSource cts = new CancellationTokenSource();
Task.Run(() =>
{
    // Long-running operation
    for (int i = 0; i < 10000; i++)
    {
        if (cts.Token.IsCancellationRequested)
        {
            // Perform cleanup or return early
            return;
        }
        // Perform some work
    }
}, cts.Token);
// To request cancellation:
// cts.Cancel();
```

Time-Based Cancellation:

You can also use `CancellationToken` with a `CancellationTokenSource` and a `Timer` to cancel a `Task` after a specified timeout. Here's an example:

```

CancellationTokenSource cts = new CancellationTokenSource();
Timer timer = new Timer(_ => cts.Cancel(), null, TimeSpan.FromSeconds(5),
Timeout.InfiniteTimeSpan);
Task.Run(() =>
{
    while (!cts.Token.IsCancellationRequested)
    {
        // Perform some work
    }
}, cts.Token);

```

Cooperative Cancellation:

If you have multiple Tasks that need to cooperate for cancellation, you can use the same CancellationTokenSource for all of them. When you cancel the CancellationTokenSource, all associated Tasks will be canceled. This can be useful for managing a group of related Tasks.

```

CancellationTokenSource cts = new CancellationTokenSource();
Task task1 = Task.Run(() => { /* Task 1 logic */ }, cts.Token);
Task task2 = Task.Run(() => { /* Task 2 logic */ }, cts.Token);
// To request cancellation for both tasks:
// cts.Cancel();

```

Cancellation in Async Methods:

If you are working with asynchronous methods, you can also pass a CancellationToken to the async method and use it to check for cancellation during its execution. This allows you to cancel async operations when needed.

```

public async Task DoAsyncOperation(CancellationToken cancellationToken)
{
    while (!cancellationToken.IsCancellationRequested)
    {
        // Perform async work
        await Task.Delay(1000);
    }
}

```

In all these scenarios, it's essential to regularly check the CancellationToken's IsCancellationRequested property within the Task's logic to detect the cancellation request and respond appropriately, whether cleaning up resources, returning early, or completing the Task gracefully.

Realtime Example to Understand Cancellation Token in C#:

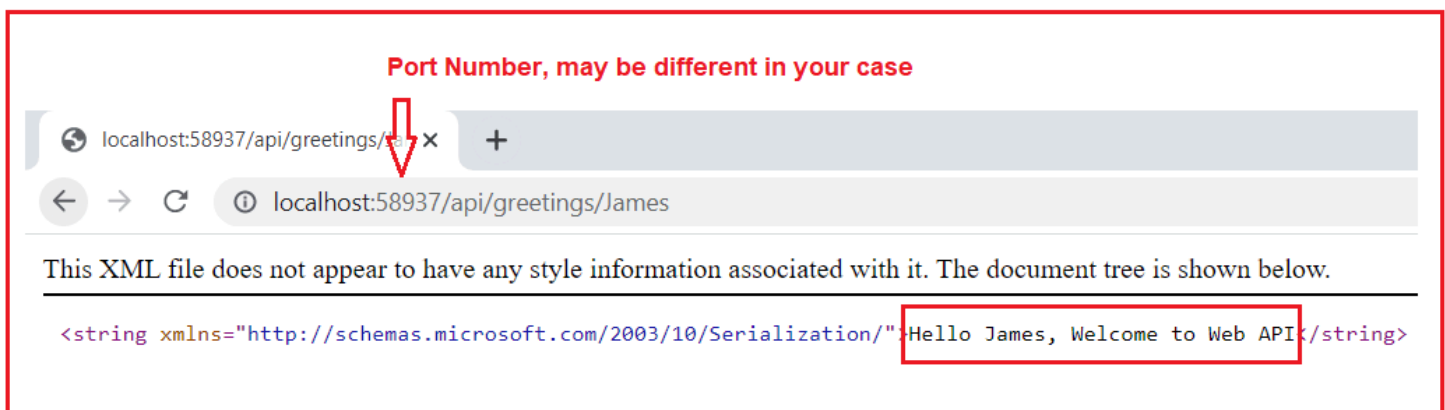
Creating ASP.NET Web API Project

Open Visual Studio and create a new ASP.NET Web API Project. If you are new to ASP.NET Web API, please look at our [ASP.NET Web API Tutorials](#). Here, we are creating an Empty Web API Project named WebAPIDemo. Once we have created the Web API Project, add a Web API Controller named

HomeController inside the Controllers folder. Once you add the HomeController, please copy and paste the following code inside it. Here, we are creating an async method that returns a string, and intentionally, we delayed the execution for 5 seconds.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Threading.Tasks;
using System.Web.Http;
namespace WebAPIDemo.Controllers
{
    public class HomeController : ApiController
    {
        //api/greetings/name
        [Route("api/greetings/{name}")]
        [HttpGet]
        public async Task<string> GetGreetings(string name)
        {
            await Task.Delay(5000);
            return $"Hello {name}, Welcome to Web API";
        }
    }
}
```

Now, run the Web API application, and you can access the GetGreetings resource using the URL **api/greetings/name** as shown in the image below. In place of a name, you can pass any values. Please note the port number; it might be different in your case.



Calling Web API from Console Application using Cancellation Token:

Now, we will make an HTTP Request to the Web API from our Console Application. Please copy the endpoint address of the Web API. And then modify the code as follows. You need to replace the port number on which your Web API application is running. In the below example, we are making an asynchronous call to the Web API. Here, please observe the GetAsync method, the second parameter of this overloaded version taking the Cancellation Token, and internally, it cancels the execution of the task after 4 seconds.

```

using System;
using System.Diagnostics;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            string Name = "James";
            SomeMethod(Name);
            Console.ReadKey();
        }
        private static async void SomeMethod(string Name)
        {
            Console.WriteLine("Some Method Started");
            using (var client = new HttpClient())
            {
                CancellationTokenSource cancellationTokenSource = new
CancellationTokenSource(4000);
                client.BaseAddress = new Uri("http://localhost:58937/");
                try
                {
                    Console.WriteLine("Some Method Calling Web API");
                    HttpResponseMessage response = await
client.GetAsync($"api/greetings/{Name}", cancellationTokenSource.Token);
                    string message = await
response.Content.ReadAsStringAsync();
                    Console.WriteLine(message);
                }
                catch (TaskCanceledException ex)
                {
                    Console.WriteLine($"Task Execution Cancelled:
{ex.Message}");
                }
                Console.WriteLine("Some Method Completed");
            }
        }
    }
}

```

Output:

```

Some Method Started
Some Method Calling Web API
Task Execution Cancelled: A task was canceled.
Some Method Completed

```

Note: Before running the Console Application, run the Web API application first.

Now, change the task cancellation time interval to 10 seconds and execute the following program.

```
using System;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            string Name = "James";
            SomeMethod(Name);
            Console.ReadKey();
        }
        private static async void SomeMethod(string Name)
        {
            Console.WriteLine("Some Method Started");
            using (var client = new HttpClient())
            {
                CancellationTokenSource cancellationTokenSource = new
CancellationTokenSource(10000);
                client.BaseAddress = new Uri("http://localhost:58937/");
                try
                {
                    Console.WriteLine("Some Method Calling Web API");
                    HttpResponseMessage response = await
client.GetAsync($"api/greetings/{Name}", cancellationTokenSource.Token);
                    string message = await
response.Content.ReadAsStringAsync();
                    Console.WriteLine(message);
                }
                catch (TaskCanceledException ex)
                {
                    Console.WriteLine($"Task Execution Cancelled:
{ex.Message}");
                }
                Console.WriteLine("Some Method Completed");
            }
        }
    }
}
```

Output:

```
Some Method Started  
Some Method Calling Web API  
"Hello James, Welcome to Web API"  
Some Method Completed
```

This time, as you can see, the task is not canceled. This is because the task was completed before 10 seconds, i.e., we got the API response before 10 seconds.

In the next article, I will discuss [Creating a Synchronous Method in C# using Tasks](#) with Examples. In this article, I explain **How to Cancel a Long-Running Task using Cancellation Token in C#** with Examples. I hope you enjoy this How to Cancel a Task in C# using Cancellation Token with Examples article.